

Instituto Tecnológico de Costa Rica

Escuela de Computación

Compiladores e Intérpretes - IC5701

Proyecto #1

Análisis Léxico y Sintáctico

Estudiantes:

Alice Arias Salazar | 2023104639

Heldyis Agüero Espinoza | 2023296812

Profesor:

Allan Rodriguez Davila

Grupo 60

Entrega: sábado 3 de octubre 2026

II Semestre 2026

Contenido

Manual de usuario	4
Requisitos necesarios	4
Paso 1. Verificar los programas necesarios	4
Obtener el proyecto desde GitHub	4
Paso 2. Abrir el repositorio en el navegador	4
Paso 3. Descargar el proyecto como ZIP	5
Paso 4. Localizar el archivo ZIP descargado	6
Paso 5. Extraer el proyecto	6
Paso 6. Identificar la carpeta correcta	6
4. Alternativa: clonar el proyecto utilizando Git	7
Paso 7. Utilizar Git en lugar de descargar el ZIP	7
5. Abrir el proyecto en Visual Studio Code	8
Paso 8. Iniciar Visual Studio Code	8
Paso 9. Seleccionar la carpeta del proyecto	8
Verificar que las librerías ya están incluidas	8
Paso 10. Abrir la carpeta Programa	8
Paso 11. Comprobar la carpeta librerías	9
Paso 12. No modificar la carpeta librerías	9
7. Verificar Java	9
Paso 13. Abrir la terminal de Visual Studio Code	9
Paso 14. Comprobar Java	9
Conocer la estructura del proyecto	10
Paso 15. Identificar los archivos principales	10
Paso 16. Comprender la función de la carpeta bin	10
Compilar el proyecto	11
Paso 17. Entrar a la carpeta de ejecutables	11
Paso 18. Ejecutar build.bat	11
Ejecutar el programa	12

Paso 19. Ejecutar run.bat	12
Paso 20. Seleccionar un archivo .cmm	13
Comprender el proceso de análisis	13
Paso 21. Análisis léxico	13
Paso 22. Generación del archivo de tokens	14
Paso 23. Análisis sintáctico	15
¿Qué hacer si aparece un error?.....	17
Paso 24. Error durante la compilación	17
Paso 25. Error relacionado con Java.....	17
Paso 26. Error relacionado con un archivo .cmm	17
Documentos de Prueba.....	17
Descripción del problema.....	19
Diseño del programa.....	20
Librerías usadas.....	22
java.io.File	22
java.io.FileDescriptor	23
java.io.FileInputStream	23
java.io.FileOutputStream	23
java.io.InputStreamReader	23
java.io.PrintStream	24
java.io.PrintWriter	24
java.lang.reflect.Field.....	24
java.lang.reflect.Modifier	24
java.nio.charset.StandardCharsets	24
java.nio.file.Path	25
java.nio.file.Paths	25
java.util.ArrayList.....	25
java.util.HashMap	25
java.util.LinkedHashSet	26

java.util.regex.Matcher	26
java.util.regex.Pattern	26
java_cup.runtime.Symbol	26
JFlex 1.9.1	27
Java CUP 11b	27
Java JDK 21.0.12.....	27
Análisis de resultados	27
Justificación de toma de decisiones	28
Bitácora	30

Manual de usuario

El proyecto se encuentra almacenado en el repositorio de GitHub del equipo:

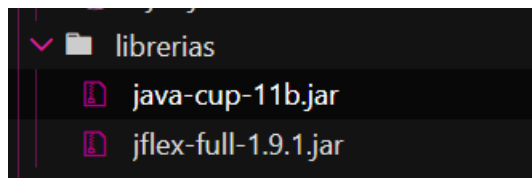
https://github.com/HeldyisAE/PY1_CompiladoresEInterpretes.git

Requisitos necesarios

Paso 1. Verificar los programas necesarios

Para ejecutar el proyecto se necesita una computadora con Windows, Visual Studio Code y Java JDK 21. También se puede utilizar Git si se desea clonar el repositorio mediante comandos, pero Git no es obligatorio si se utiliza la opción de descargar el proyecto como ZIP

No es necesario instalar JFlex ni Java CUP manualmente. Las versiones utilizadas por el proyecto ya están incluidas dentro del repositorio en la carpeta Programa\librerías.



La estructura de esa carpeta contiene las librerías necesarias para que los archivos ejecutables del proyecto puedan encontrarlas automáticamente durante el proceso de compilación.

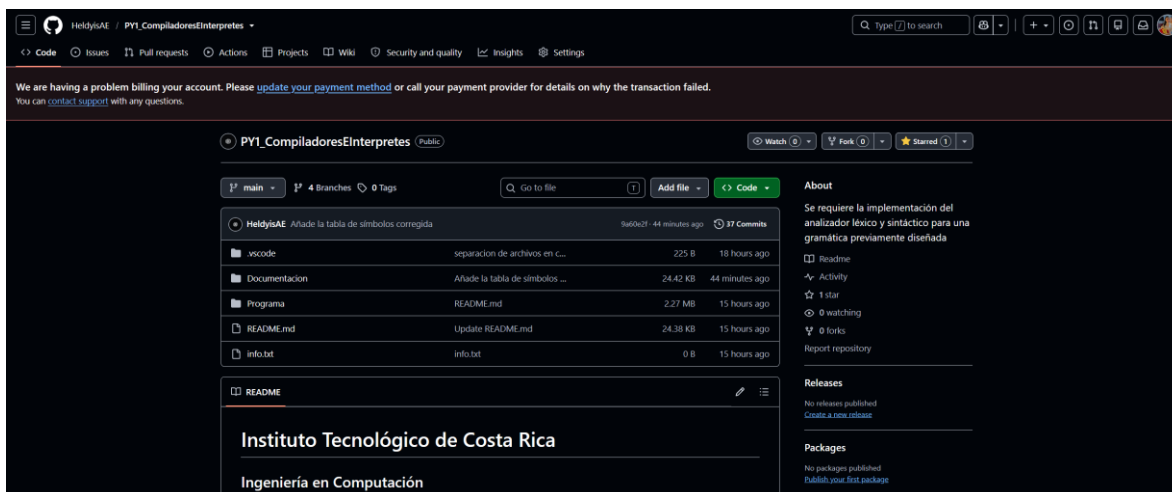
Obtener el proyecto desde GitHub

Paso 2. Abrir el repositorio en el navegador

Primero se debe abrir un navegador web como Google Chrome, Microsoft Edge o Mozilla Firefox. En la barra de direcciones se debe ingresar:

https://github.com/HeldyisAE/PY1_CompiladoresEInterpretes.git

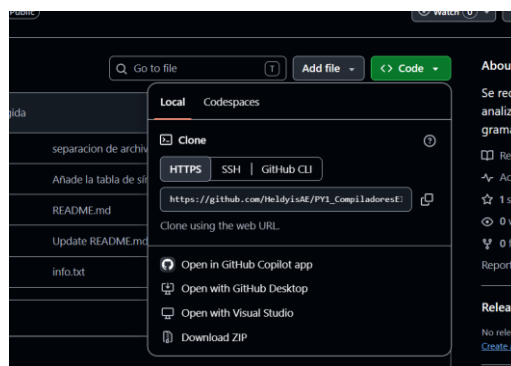
y presionar Enter. Se abrirá la página del repositorio del proyecto en GitHub.



En esta página se pueden observar las carpetas y archivos que forman parte del proyecto.

Paso 3. Descargar el proyecto como ZIP

Para una persona que no conoce Git, esta es la opción más sencilla. En la página del repositorio se debe localizar el botón verde que dice Code. Al presionarlo aparecerá un menú con diferentes opciones. Dentro de este menú se debe seleccionar: Download ZIP



El navegador comenzará a descargar un archivo comprimido que contiene una copia completa del proyecto. El archivo tendrá un nombre similar a:

PY1_CompiladoresEInterpretes-main.zip

El nombre exacto puede variar dependiendo de cómo esté configurado el repositorio.

Paso 4. Localizar el archivo ZIP descargado

Normalmente el archivo ZIP se guarda automáticamente en la carpeta Descargas de Windows. Para encontrarlo se puede abrir el explorador de archivos de Windows y seleccionar: Descargas. Dentro de esta carpeta se debe buscar: PY1_CompiladoresEInterpretes-main.zip. Este archivo todavía está comprimido, por lo que no se recomienda intentar abrir el proyecto directamente desde el ZIP.

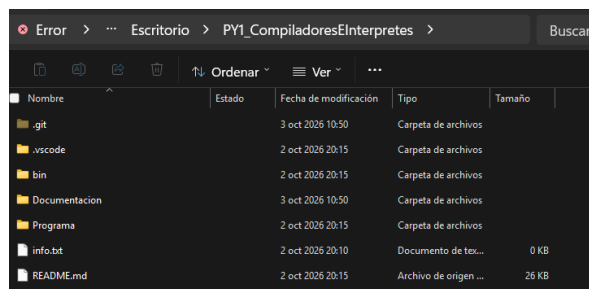
Paso 5. Extraer el proyecto

Para extraer el proyecto se debe hacer clic derecho sobre el archivo: PY1_CompiladoresEInterpretes-main.zip. En el menú que aparece se debe seleccionar la opción: Extraer todo... Windows mostrará una ventana donde se puede seleccionar la ubicación donde se desea colocar el proyecto. Para facilitar el procedimiento, se puede seleccionar el Escritorio. Después se debe presionar Extraer. Windows creará una carpeta con el proyecto completo. La ubicación puede quedar similar a:

C:\Users\NombreUsuario\Desktop\PY1_CompiladoresEInterpretes-main

Paso 6. Identificar la carpeta correcta

Después de extraer el ZIP es importante entrar en la carpeta que realmente contiene el proyecto. Al abrir la carpeta extraída se deben encontrar elementos similares a:



La carpeta que contiene estos elementos es la carpeta principal del proyecto. Si al abrir la carpeta solamente aparece otra carpeta llamada

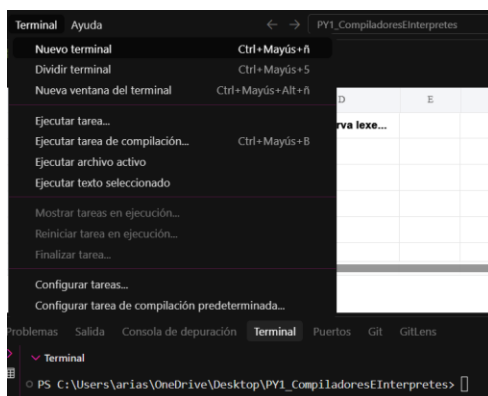
PY1_CompiladoresEInterpretes-main, se debe entrar en ella hasta encontrar Programa, Documentacion y README.md.

Esto es importante porque Visual Studio Code debe abrir la carpeta principal del proyecto, no solamente el archivo ZIP ni una carpeta superior que no contenga los archivos del proyecto.

4. Alternativa: clonar el proyecto utilizando Git

Paso 7. Utilizar Git en lugar de descargar el ZIP

La descarga mediante ZIP no es la única forma de obtener el proyecto. Si el usuario ya tiene Git instalado, también puede clonar directamente el repositorio. Esta alternativa permite mantener una conexión con el repositorio y facilita obtener posteriormente actualizaciones mediante Git. Primero se debe abrir Visual Studio Code y seleccionar:



Después se debe escribir:

```
git clone https://github.com/HeldyisAE/PY1_CompiladoresEInterpretes.git
```

Al presionar Enter, Git descargará el proyecto y creará una carpeta llamada: PY1_CompiladoresEInterpretes. La principal diferencia es que con Download ZIP se obtiene una copia comprimida que posteriormente se extrae, mientras que con git clone Git crea directamente una copia local del repositorio.

Nota: Para este manual se recomienda Download ZIP como método principal, porque es el procedimiento más sencillo para una persona que nunca ha utilizado Git.

5. Abrir el proyecto en Visual Studio Code

Paso 8. Iniciar Visual Studio Code

Después de descargar y extraer el proyecto, se debe abrir Visual Studio Code. No se debe crear un proyecto Java nuevo. El proyecto ya está creado y contiene todos los archivos necesarios.

Paso 9. Seleccionar la carpeta del proyecto

En la ventana que aparece se debe buscar el Escritorio y seleccionar la carpeta:

PY1_CompiladoresEInterpretes-main

o el nombre correspondiente de la carpeta que se obtuvo después de extraer el ZIP.

Después se debe presionar:

Select Folder

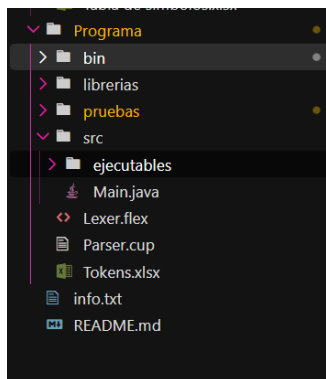
Visual Studio Code cargará todos los archivos del proyecto. En el lado izquierdo aparecerá el explorador de archivos.

Verificar que las librerías ya están incluidas

Paso 10. Abrir la carpeta Programa

Dentro del explorador de Visual Studio Code se debe localizar: Programa Al abrirla se podrán observar las carpetas y archivos principales del programa.

Entre ellos debe encontrarse:



Paso 11. Comprobar la carpeta librerías

Dentro de Programa se debe abrir: librerías. Esta carpeta contiene las librerías que utiliza el proyecto. Entre las herramientas incluidas se encuentran los archivos correspondientes a: JFlex 1.9.1 Y Java CUP 11b. Por esta razón, el usuario no tiene que entrar a Internet para descargar nuevamente estas librerías. El proyecto ya fue preparado para utilizar las versiones incluidas dentro del repositorio. El archivo build.bat utiliza las librerías almacenadas en esta carpeta durante el proceso de compilación.

Paso 12. No modificar la carpeta librerías

Aunque las librerías ya estén incluidas, el usuario no debe modificar sus archivos. No se deben cambiar sus nombres, moverlos a otra carpeta ni reemplazarlos por versiones descargadas de Internet. El proyecto está configurado para buscar las librerías en: Programa\librerías Si se cambia esta ubicación, los archivos build.bat y run.bat podrían dejar de encontrar las dependencias necesarias.

7. Verificar Java**Paso 13. Abrir la terminal de Visual Studio Code**

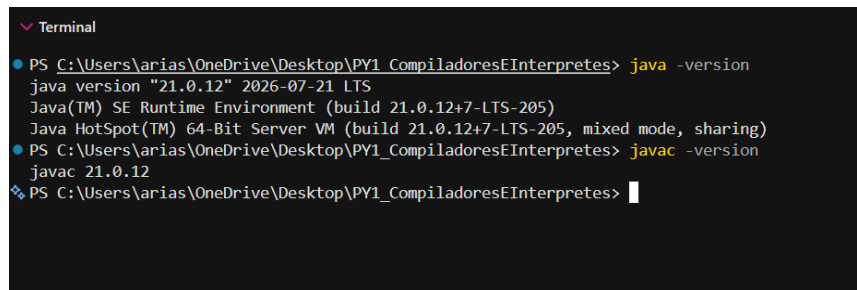
Aunque las librerías de JFlex y Java CUP ya estén incluidas, Java sí debe estar instalado en la computadora. En Visual Studio Code se debe seleccionar:

Terminal - New Terminal

En la parte inferior aparecerá una terminal.

Paso 14. Comprobar Java

En la terminal se debe escribir: java -version y presionar Enter. Después se debe escribir: javac -version y presionar Enter. El segundo comando es especialmente importante porque comprueba que el compilador de Java está disponible. Un resultado correcto puede ser similar a:



```
Terminal
PS C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes> java -version
java version "21.0.12" 2026-07-21 LTS
Java(TM) SE Runtime Environment (build 21.0.12+7-LTS-205)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.12+7-LTS-205, mixed mode, sharing)
PS C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes> javac -version
javac 21.0.12
PS C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes> |
```

El número exacto puede cambiar si se utiliza otra actualización de Java 21. Si ambos comandos funcionan, la computadora está preparada para continuar con el proceso.

Conocer la estructura del proyecto

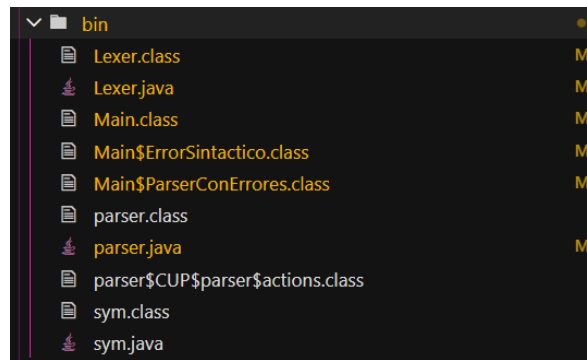
Paso 15. Identificar los archivos principales

La carpeta Programa contiene los archivos principales del proyecto. El archivo:

- **Lexer.flex:** contiene las reglas utilizadas por JFlex para generar el analizador léxico.
- **Parser.cup:** contiene las reglas de la gramática utilizadas por Java CUP para generar el analizador sintáctico.
- **src\Main.java:** contiene el programa principal que coordina el análisis.
- **src\ejecutables:** contiene los archivos que facilitan la compilación y ejecución. Dentro de esta carpeta se encuentran:
 - build.bat
 - run.bat
- **pruebas:** contiene los archivos .cmm utilizados como entradas de prueba.
- **Librerías:** contiene JFlex y Java CUP.
- **Bin:** contiene archivos generados y compilados.

Paso 16. Comprender la función de la carpeta bin

La carpeta bin es una carpeta de salida utilizada durante la compilación. Los archivos que aparecen allí son generados automáticamente por el proceso de construcción del proyecto. Por ejemplo, pueden aparecer archivos como:



El usuario no debe editar estos archivos manualmente. Tampoco debe moverlos, renombrarlos, eliminarlos ni crear archivos nuevos dentro de bin. Si se modifica Lexer.flex o Parser.cup, los archivos correspondientes deben regenerarse ejecutando nuevamente el proceso de compilación. Por esta razón, bin debe tratarse como una carpeta de archivos generados automáticamente.

Compilar el proyecto

Paso 17. Entrar a la carpeta de ejecutables

Los archivos ejecutables no están directamente dentro de Programa. Se encuentran en: Programa\src\ejecutables Por lo tanto, en la terminal se debe ingresar:

```
terminal
PS C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes> cd Programa
PS C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes\Programa> cd src
PS C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes\Programa\src> cd ejecutables
PS C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes\Programa\src\ejecutables>
```

Esta es la ubicación correcta para ejecutar los archivos .bat.

Paso 18. Ejecutar build.bat

Desde la carpeta Programa\src\ejecutables se debe escribir:

```

PS C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes\Programa\src\ejecutables> .\build.bat

=====
COMPILEACION DEL PROYECTO
=====

Proyecto : C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes\Programa\src\ejecutables\...\
JFlex    : C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes\Programa\src\ejecutables\...\librerias\jflex-full-1.9.1.jar
CUP      : C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes\Programa\src\ejecutables\...\librerias\java-cup-11b.jar
Runtime  : C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes\Programa\src\ejecutables\...\librerias\java-cup-11b.jar
Bin      : C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes\Programa\src\ejecutables\...\bin

Limpiando archivos generados anteriores...
Limpieza de bin completada.

Generando Parser con CUP...
Warning : *** Production "Principal ::= PRINCIPAL " never reduced
----- CUP v0.11b 20150326 (SVN rev 63) Parser Generation Summary -----
  0 errors and 1 warning
  54 terminals, 98 non-terminals, and 211 productions declared,
  producing 394 unique parse states.
  0 terminals declared but not used.
  0 non-terminals declared but not used.
  0 productions never reduced.
  0 conflicts detected (0 expected).
  Code written to "parser.java", and "sym.java".
----- (CUP v0.11b 20150326 (SVN rev 63))
CUP generado correctamente.

Generando Lexer con JFlex...
Reading "Lexer.flex"
Constructing NFA : 997 states in NFA
Converting NFA to DFA :
.....
253 states before minimization, 146 states in minimized DFA
Writing code to "C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes\Programa\src\ejecutables\...\bin\Lexer.java"
JFlex generado correctamente.

Compilando archivos Java...
Note: C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes\Programa\src\ejecutables\...\src\Main.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

```

Después se presiona Enter. El archivo build.bat realiza automáticamente el proceso de compilación.

Primero localiza las librerías que ya están incluidas en Programa\librerias. Después utiliza Java CUP para generar el parser, utiliza JFlex para generar el lexer y finalmente utiliza javac para compilar los archivos Java. Por lo tanto, el usuario no necesita descargar JFlex ni Java CUP ni ejecutar manualmente cada uno de estos comandos.

Ejecutar el programa

Paso 19. Ejecutar run.bat

Cuando la compilación haya terminado correctamente, se debe permanecer en:

```

PS C:\Users\arias\OneDrive\Desktop\PY1_CompiladoresEInterpretes\Programa\src\ejecutables> .\run.bat

Carpeta de pruebas : pruebas\

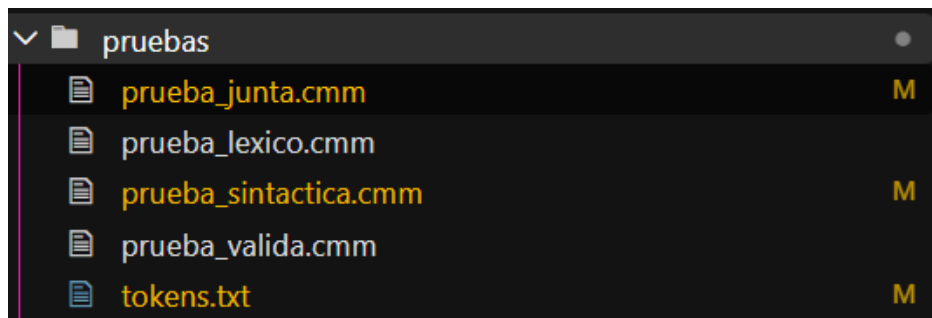
Nombre del archivo .cmm: 

```

y se presiona Enter. Este archivo inicia el programa y solicita al usuario el archivo .cmm que desea analizar.

Paso 20. Seleccionar un archivo .cmm

Los archivos de prueba están ubicados en:



Dentro de esta carpeta se encuentran los archivos fuente utilizados para probar el analizador. Estos archivos tienen la extensión: .cmm

Cuando run.bat solicite el nombre del archivo, se debe introducir el nombre del archivo que se desea probar.

Comprender el proceso de análisis

Paso 21. Análisis léxico

Después de seleccionar el archivo .cmm, el programa comienza el análisis léxico. JFlex lee el archivo fuente y reconoce los elementos definidos en Lexer.flex. Estos elementos pueden ser palabras reservadas, identificadores, números, cadenas, caracteres, operadores, delimitadores y comentarios. Cada elemento reconocido se transforma en un token que posteriormente será utilizado por el analizador sintáctico.

```

ANALIZADOR LÉXICO Y SINTÁCTICO

Archivo fuente      : pruebas\prueba_lexico.cmm
Archivo de tokens   : pruebas\tokens.txt

--- FASE 1 • ANÁLISIS LÉXICO ---

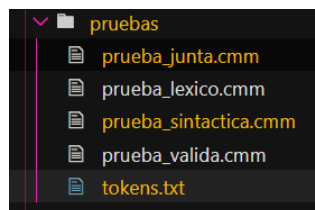
Tokens reconocidos  : 709
Errores léxicos     : 110

Nº  LÍNEA  COL  DESCRIPCIÓN
---
01   27    31  carácter no reconocido '.'
02   28    29  carácter no reconocido '.'
03   29    26  literal float incompleto '123.'
04   29    30  carácter no reconocido '.'
05   30    26  carácter no reconocido '.'
06   31    26  literal float incompleto '123.'
07   32    30  carácter no reconocido '.'
08   33    29  carácter no reconocido '.'
09   35    42  carácter no reconocido '.'
10   36    25  identificador inválido '123abc'
11   42     9  identificador inválido '_variable1'
12   43     9  identificador inválido '_variable2'
13   44     9  identificador inválido '2variable'
14   45     9  identificador inválido '123nombre'
15   47    15  carácter no reconocido '@'
16   48    15  carácter no reconocido '#'
17   49    15  carácter no reconocido '$'
18   50    15  carácter no reconocido '.'
19   58    30  literal string sin cerrar
20   59    21  carácter no reconocido '@'
21   59    25  literal string sin cerrar
22   60    21  carácter no reconocido '#'
23   60    25  literal string sin cerrar
24   61    21  carácter no reconocido '$'
25   61    25  literal string sin cerrar
26   62    29  literal string sin cerrar
27   63    13  literal string sin cerrar
28   71    22  literal char vacío ''

```

Paso 22. Generación del archivo de tokens

Durante el análisis se genera el archivo:



Este archivo contiene los tokens reconocidos. El usuario puede abrirlo desde Visual Studio Code después de ejecutar el programa para revisar los resultados.

Este archivo sí es un archivo de salida de la ejecución y no debe confundirse con los archivos generados dentro de bin.

tokens.txt M X

Programa > pruebas > tokens.txt

Usted, hace 55 segundos | autor (Usted)

	LINEA	COLUMNA	TOKEN	LEXEMA
1				
2				
3	10	1	VAL	val
4	10	5	INT	int
5	10	9	ID	entero1
6	10	17	ASIGN	=
7	10	19	LITERAL_INT	0
8	10	20	TERMINATOR	»
9	11	1	VAL	val
10	11	5	INT	int
11	11	9	ID	entero2
12	11	17	ASIGN	=
13	11	19	LITERAL_INT	1
14	11	20	TERMINATOR	»
15	12	1	VAL	val
16	12	5	INT	int
17	12	9	ID	entero3
18	12	17	ASIGN	=
19	12	19	LITERAL_INT	10
20	12	21	TERMINATOR	»
21	13	1	VAL	val
22	13	5	INT	int
23	13	9	ID	entero4
24	13	17	ASIGN	=
25	13	19	LITERAL_INT	123456789
26	13	28	TERMINATOR	»
27	14	1	VAL	val
28	14	5	FLOAT	float
29	14	11	ID	decimal1
30	14	20	ASIGN	=

```

709 183 10 LITERAL_STRING "Fin de prueba lexica dificil"
710 183 41 PARENCL :@
711 185 1 CLBLOCK :? Usted, hace 47 segundos • Cambios sin commit
712
713 ERRORES LEXICOS (110)
714 -----
715 Error léxico en línea 27, columna 31: carácter no reconocido '.'
716 Error léxico en línea 28, columna 29: carácter no reconocido '.'
717 Error léxico en línea 29, columna 26: literal float incompleto '123.'
718 Error léxico en línea 29, columna 30: carácter no reconocido '.'
719 Error léxico en línea 30, columna 26: carácter no reconocido '.'
720 Error léxico en línea 31, columna 26: literal float incompleto '123.'
721 Error léxico en línea 32, columna 30: carácter no reconocido '.'
722 Error léxico en línea 33, columna 29: carácter no reconocido '.'
723 Error léxico en línea 35, columna 42: carácter no reconocido '.'
724 Error léxico en línea 36, columna 25: identificador inválido '123abc'
725 Error léxico en línea 42, columna 9: identificador inválido '_variable1'
726 Error léxico en línea 43, columna 9: identificador inválido '_variable2'
727 Error léxico en línea 44, columna 9: identificador inválido '2variable'
728 Error léxico en línea 45, columna 9: identificador inválido '123nombre'
729 Error léxico en línea 47, columna 15: carácter no reconocido '@'
730 Error léxico en línea 48, columna 15: carácter no reconocido '#'
731 Error léxico en línea 49, columna 15: carácter no reconocido '$'
732 Error léxico en línea 50, columna 15: carácter no reconocido '.'
733 Error léxico en línea 58, columna 30: literal string sin cerrar
734 Error léxico en línea 59, columna 21: carácter no reconocido '@'

```

Paso 23. Análisis sintáctico

Después del análisis léxico, los tokens son enviados al analizador sintáctico generado por Java CUP. El parser comprueba que los tokens estén organizados de acuerdo con las reglas establecidas en Parser.cup. Si el archivo cumple con las reglas de la gramática, el programa indicará que el análisis fue correcto.

```

ANALIZADOR LÉXICO Y SINTÁCTICO

Archivo fuente      : pruebas\prueba_valida.cmm
Archivo de tokens   : pruebas\tokens.txt

--- FASE 1 - ANÁLISIS LÉXICO ---

Tokens reconocidos : 1021
Errores léxicos    : 0

Análisis léxico correcto.

--- FASE 2 - ANÁLISIS SINTÁCTICO ---

Análisis sintáctico correcto.

--- RESULTADO FINAL ---

Tokens reconocidos : 1021
Errores léxicos    : 0
Errores sintácticos : 0
-----
Errores totales    : 0
Tiempo de ejecución : 145 ms

ARCHIVO CORRECTO
No se encontraron errores léxicos ni sintácticos.

```

Si encuentra una estructura incorrecta, mostrará información sobre el error sintáctico.

```

--- FASE 2 - ANÁLISIS SINTÁCTICO ---

ERROR 01 - línea 10, columna 8

10 | val int»
    | ^
Se encontró : '»' (TERMINATOR)
Se esperaba : identificador

ERROR 02 - línea 11, columna 19

11 | val int variable t»
    | ^
Se encontró : '»' (TERMINATOR)
Se esperaba : -, Σ, ε:, true, false, número entero, ...

ERROR 03 - línea 12, columna 9

12 | val int t 10»
    | ^
Se encontró : 't' (ASIGN)
Se esperaba : identificador

```

```

El archivo contiene errores sintácticos.

--- RESULTADO FINAL ---

Tokens reconocidos : 707
Errores léxicos    : 0
Errores sintácticos : 51
-----
Errores totales    : 51
Tiempo de ejecución : 488 ms

ARCHIVO CON ERRORES
Corrige los errores indicados y vuelve a ejecutar.

```

¿Qué hacer si aparece un error?

Paso 24. Error durante la compilación

Si build.bat muestra un error, lo primero que se debe revisar es que se esté ejecutando desde: Programa\src\ejecutables También se debe comprobar que existan: Programa\librerías, Programa\bin, Programa\Lexer.flex, Programa\Parser.cup

No se deben descargar otras versiones de JFlex o Java CUP para solucionar el problema sin antes revisar la configuración del proyecto, porque las versiones necesarias ya están incluidas en el repositorio.

Paso 25. Error relacionado con Java

Si aparece un mensaje indicando que java o javac no se reconoce, el problema está relacionado con la instalación o configuración del JDK. Se debe comprobar nuevamente: java -version y javac -version

Ambos comandos deben funcionar antes de intentar nuevamente ejecutar build.bat.

Paso 26. Error relacionado con un archivo .cmm

Si el programa muestra un error indicando una línea, columna, token o lexema, significa que el archivo .cmm contiene algún elemento que no cumple con las reglas del lenguaje. En este caso se debe revisar el archivo de prueba indicado y observar la línea y columna reportadas por el analizador. No se debe modificar bin para intentar solucionar este tipo de errores. Los errores del archivo .cmm deben corregirse en el archivo fuente correspondiente.

Documentos de Prueba

```
val int a ← 10»int suma €: int x, int y :» ¿: return x + y» :?int calcular
€: int x, int y :» ¿: val int resultado ← €: x + y :» * 2» return resultado»
:?int calcularTres €: int x, int y, int z :» ¿: val int resultado ← €: x
+ y :» * z» return resultado»:?float calcularDecimal €: float x, float y
:» ¿: return x + y» :?bool esPositivo €: int numero :» ¿: if €: numero >
0 :» ¿: return true» :? else ¿: return false» :? :?void principal €: :» ¿:
```

```

val int a ← 10» val float x ← 10.5» val float y ← 20.25» val bool activo
← true» val bool terminado ← false» val char inicial ← 'A'» val char final
← 'Z'» val string nombre ← "Alice"» val string mensaje ← "Programa
completamente valido"» val int suma1 ← a + b» val int suma2 ← a + b + c»
val int resta1 ← c - a» val int multiplicacion1 ← a * b» val int division1
← c / a» val int division2 ← c // a» val int modulo1 ← c mod a» val int
potencia1 ← 2 pot 3» val int negativo1 ← -1» val int agrupada1 ← ε: a + b
:ε * c» val int agrupada2 ← ε: a + b :ε * ε: c + d :ε» val int agrupada3
← ε: ε: a + b :ε * c :ε + d» val int agrupada4 ← ε: a * ε: b + c :ε - d
:ε» val int agrupada5 ← ε: ε: a + b :ε * ε: c - d :ε :ε» val int potencia1
← 2 pot 3» val int potencia2 ← 2 pot ε: 3 + 1 :ε» val int potencia3 ← ε:
2 pot 3 :ε + 10» val int potencia4 ← ε: ε: 2 pot 3 :ε + ε: 4 pot 2 :ε :ε» val
int potencia5 ← a pot 2» val int resultado1 ← suma ε: 10, 20 :ε» val int
resultado2 ← suma ε: a, b :ε» val int resultado3 ← suma ε: suma ε: 1, 2 :ε,
suma ε: 3, 4 :ε :ε» val int resultado4 ← suma ε: suma ε: 1, 2 :ε, suma ε: 3,
4 :ε :ε» val int resultado5 ← calcular ε: suma ε: 1, 2 :ε, suma ε: 3, 4 :ε :ε» val
int expresion1 ← suma ε: 10, 20 :ε + 30» val int expresion2 ← ε: suma ε: 10,
20 :ε + suma ε: 30, 40 :ε :ε» val int expresion3 ← ε: suma ε: suma ε: 1, 2
:ε, suma ε: 3, 4 :ε :ε * 2 :ε» val int expresion4 ← calcular ε: suma ε: 1, 2
:ε, ε: 3 * 4 :ε :ε» val bool logica1 ← activo» val bool logica2 ← terminado» val
int numero1 ← 10» val int numero2 ← 20» val int numero3 ← 30» val int profunda1
← ε: numero1 + ε: numero2 * numero3 :ε :ε» val int profunda2 ← ε: ε: numero1
+ numero2 :ε * ε: numero3 - 5 :ε :ε» val int profunda3 ← ε: ε: numero1 +
numero2 :ε * ε: numero3 + 10 :ε :ε» val int profunda4 ← ε: suma ε: numero1,
numero2 :ε + suma ε: numero2, numero3 :ε :ε» val int potenciaProfunda1 ← 2
pot ε: 3 + 1 :ε» val int potenciaProfunda2 ← ε: 2 pot 3 :ε + 10» val int
potenciaProfunda3 ← ε: ε: 2 pot 3 :ε + ε: 4 pot 2 :ε :ε» if ε: numero1 > 0 :ε
¿: print ε: "numero1 es positivo" :ε :? if ε: ε: numero1 > 0 :ε λ ε: numero2 >
0 :ε :ε ¿: print ε: "Los dos son positivos" :ε :? if ε: ε: numero1 > 100 :ε 0
ε: numero2 > 0 :ε :ε ¿: print ε: "Al menos una condicion es verdadera" :ε :? if
Σ ε: numero1 < 0 :ε ¿: print ε: "numero1 no es negativo" :ε :? if ε: numero1
> numero2 :ε ¿: print ε: "numero1 es mayor" :ε :? else ¿: print ε: "numero2
es mayor o igual" :ε :? while ε: numero2 > 0 :ε ¿: val int calculo ← ε: numero1

```

```

+ numero2:»numero2 + numero2 - 1»:?»for e: i + 0» i < 10» i++ :» ¿:print
e: i :»?»if e: numero1 > 0 :» ¿:while e: numero2 > 0 :» ¿:if e: numero3 >
0 :» ¿:val int calculoAnidado + e:numero1 +e:numero2 * numero3:»»print
e: calculoAnidado :»?»numero2 + numero2 - 1»:?» :?»for e: j + 0» j < 5» j++
:» ¿: if e: j > 2 :» ¿:print e: j :»?:?»numero1++numero1++numero2--numero3--
numero3++numero3++read e: numero :»read e: numero :»write e: numero
:»write e: numero :»mostrar e: mensaje :»»val int final1 + suma
e:numero1,numero2:»»val int final2 + calcular e:suma e: 1, 2 :»,suma e: 3,
4 :»:»»val int final3 + e:final1 + final2:»»print e: "Programa
completamente valido" :»:?»

```

Descripción del problema.

Los dispositivos electrónicos y sistemas embebidos requieren mecanismos de configuración que permitan definir su comportamiento de una manera sencilla, eficiente y adecuada para las características de este tipo de sistemas. Debido a esto, surge la necesidad de contar con lenguajes de programación ligeros que permitan representar instrucciones de configuración sin requerir la complejidad de lenguajes de propósito general. En este proyecto se plantea el desarrollo de un nuevo lenguaje de programación imperativo, diseñado para realizar configuraciones básicas en chips y sistemas embebidos.

El lenguaje debe permitir al programador definir variables, expresiones, operaciones, funciones, estructuras de control y otras instrucciones necesarias para representar un programa. Para poder procesar programas escritos en este lenguaje es necesario construir las primeras etapas de un compilador. En este primer proyecto se desarrollan específicamente el analizador léxico y el analizador sintáctico, tomando como base una gramática definida mediante BNF.

El programa desarrollado recibe como entrada un archivo fuente con extensión. cmm. Este archivo contiene un programa escrito utilizando las reglas y elementos definidos para el nuevo lenguaje. Antes de determinar si el programa es válido, es necesario identificar cada uno de los elementos que lo componen. Esta tarea corresponde al análisis léxico.

Por lo tanto, el problema que aborda este proyecto consiste en construir un sistema capaz de recibir un programa escrito en un lenguaje de programación propio, identificar correctamente sus componentes léxicos, comprobar que estos cumplen con las reglas sintácticas definidas y reportar los errores encontrados de manera comprensible, permitiendo continuar el análisis cuando sea posible.

El resultado de estas etapas constituye la base para las siguientes fases de un compilador. Aunque en este proyecto no se realiza la generación de código ejecutable, el análisis léxico y sintáctico permite determinar si un programa está correctamente formado de acuerdo con las reglas establecidas para el lenguaje.

Diseño del programa

El programa fue diseñado como un analizador léxico y sintáctico para el lenguaje definido en el proyecto. Se decidió separar las responsabilidades de cada etapa para facilitar la organización, el mantenimiento y la comprensión del código. El análisis léxico se encarga de identificar los elementos individuales del programa fuente y convertirlos en tokens, mientras que el análisis sintáctico utiliza esos tokens para comprobar que la estructura del programa cumple con las reglas establecidas en la gramática.

Para el análisis léxico se utilizó JFlex, debido a que permite definir mediante expresiones regulares los patrones correspondientes a los diferentes elementos del lenguaje. El analizador reconoce palabras reservadas, identificadores, tipos de datos, números, caracteres, cadenas, operadores y delimitadores. También se incorporaron reglas para ignorar espacios y comentarios, de manera que estos elementos no interfieran con el análisis del programa. Cuando aparece un carácter que no pertenece al lenguaje, se genera un error léxico indicando información como la línea, la columna y el carácter encontrado.

Para el análisis sintáctico se utilizó Java CUP. La gramática se definió mediante producciones que describen la estructura válida del lenguaje, incluyendo declaraciones, asignaciones, expresiones, estructuras de control, funciones, arreglos, entrada y salida, y el programa principal. La utilización de producciones

separadas permite controlar mejor la estructura del lenguaje y reducir posibles ambigüedades durante el análisis.

Una decisión importante de diseño fue separar las expresiones en diferentes niveles de la gramática. Esta estructura permite controlar la precedencia y asociatividad de los operadores, de manera que expresiones que contienen varios operadores sean interpretadas en el orden correspondiente. Por ejemplo, una expresión que combina suma y multiplicación puede ser analizada respetando que la multiplicación tenga una prioridad mayor que la suma. También se contemplan expresiones agrupadas para permitir que el programador modifique explícitamente el orden mediante los delimitadores establecidos por el lenguaje.

El programa también incorpora un mecanismo de recuperación de errores sintácticos mediante Panic Mode. Cuando el analizador encuentra un error, utiliza puntos de sincronización definidos en la gramática para descartar tokens hasta encontrar una posición desde la cual pueda continuar el análisis. Esta decisión permite que el programa no tenga que detenerse inmediatamente ante el primer error sintáctico, sino que pueda continuar procesando el archivo y reportar otros errores encontrados posteriormente.

El flujo general del programa sigue una secuencia definida. Primero se recibe como entrada un archivo con extensión .cmm. Después, JFlex procesa el contenido del archivo y genera los tokens correspondientes. Estos tokens son enviados al analizador sintáctico generado con Java CUP, que verifica si la secuencia cumple con las producciones de la gramática el programa muestra los resultados del análisis y registra los tokens reconocidos en el archivo tokens.txt. Cuando se encuentran errores, se muestran datos que permiten localizar el problema dentro del archivo fuente.

En cuanto a los algoritmos utilizados, el análisis léxico se basa en el reconocimiento de patrones mediante expresiones regulares y en el procesamiento secuencial de los caracteres del archivo de entrada. El analizador avanza sobre el texto identificando el patrón correspondiente a cada elemento y generando el token

asociado. Este proceso permite transformar el código fuente en una secuencia de tokens que puede ser utilizada por la siguiente etapa.

El análisis sintáctico utiliza el algoritmo de análisis generado por Java CUP a partir de la gramática definida en Parser.cup. El parser recibe los tokens producidos por el analizador léxico y determina si pueden derivarse utilizando las producciones de la gramática. De esta manera, se comprueba que las instrucciones, declaraciones y expresiones estén estructuradas correctamente de acuerdo con las reglas del lenguaje.

Para la recuperación de errores sintácticos se utiliza el enfoque de Panic Mode. Este procedimiento consiste en descartar tokens hasta alcanzar un símbolo o punto de sincronización que permita continuar con el análisis. En el proyecto, este mecanismo se complementa con las producciones de error definidas en la gramática, permitiendo reportar los errores encontrados sin abandonar inmediatamente todo el proceso.

Y de manera para no cometer errores a la hora de ejecutar los archivos decidió automatizar la generación y ejecución del proyecto mediante los archivos build.bat y run.bat. El archivo build.bat se encarga de ejecutar las herramientas necesarias para generar el analizador léxico y sintáctico y compilar los archivos Java. El archivo run.bat facilita la ejecución del programa y permite seleccionar un archivo .cmm para realizar las pruebas. Esta automatización evita que el usuario tenga que ejecutar manualmente cada comando de JFlex, Java CUP y Java, haciendo que el proceso de compilación y ejecución sea más sencillo y menos propenso a errores.

Librerías usadas

java.io.File

La clase File se utiliza en Main.java para representar y manipular archivos y directorios del proyecto. Se utiliza principalmente para comprobar la existencia de los archivos fuente .cmm, trabajar con las rutas de entrada y salida y verificar que los archivos necesarios estén disponibles antes de iniciar el análisis. Se eligió

porque forma parte de las librerías estándar de Java y proporciona una forma sencilla de trabajar con archivos sin necesidad de instalar dependencias adicionales.

java.io.FileDescriptor

La clase FileDescriptor se utiliza en Main.java para trabajar con los descriptores asociados a los flujos de entrada y salida del sistema. Su uso permite manejar de manera controlada recursos relacionados con archivos y con la salida estándar del programa. Se eligió porque permite trabajar directamente con los recursos de entrada y salida proporcionados por Java y es compatible con las clases de java.io utilizadas en el proyecto.

java.io.FileInputStream

FileInputStream se utiliza para leer información desde archivos mediante un flujo de bytes. En este proyecto forma parte del proceso utilizado para acceder al archivo fuente .cmm que será analizado. Se eligió porque permite abrir archivos directamente desde el sistema y trabajar con su contenido como flujo de entrada, lo cual resulta adecuado para posteriormente convertir los datos a caracteres utilizando InputStreamReader.

java.io.FileOutputStream

utiliza para crear y escribir información en archivos mediante un flujo de salida. En el proyecto se utiliza para generar archivos de resultados producidos durante la ejecución, especialmente información relacionada con el análisis y los tokens. Se eligió porque permite escribir directamente sobre archivos del sistema y trabajar juntamente con otras clases de salida como PrintWriter.

java.io.InputStreamReader

se usa convertir un flujo de bytes obtenido mediante FileInputStream en un flujo de caracteres. En este proyecto es importante porque el archivo fuente contiene texto que debe ser leído correctamente por el analizador léxico. Se eligió porque permite especificar la codificación utilizada para interpretar los caracteres y trabajar adecuadamente con archivos escritos en UTF-8.

java.io.PrintStream

Envía información hacia una salida, principalmente la consola. En este proyecto permite mostrar al usuario los resultados del análisis léxico y sintáctico, mensajes del proceso de ejecución y los errores encontrados. Se eligió porque proporciona métodos sencillos para imprimir texto y valores en la consola, facilitando la presentación de los resultados del programa.

java.io.PrintWriter

se utiliza para escribir texto de manera sencilla en archivos. En este proyecto es especialmente importante para generar el archivo tokens.txt, donde se almacenan los tokens reconocidos durante el análisis léxico. Se eligió porque permite escribir líneas de texto de forma sencilla y controlada, lo cual resulta adecuado para generar archivos de resultados legibles por el usuario.

java.lang.reflect.Field

Field pertenece al mecanismo de reflexión de Java y permite acceder dinámicamente a los campos definidos dentro de una clase. En este proyecto se utiliza para consultar la información de los símbolos generados automáticamente por CUP en sym.java. Se eligió porque permite obtener dinámicamente los nombres y valores de los tokens sin tener que mantener manualmente una lista de todas las constantes generadas por CUP.

java.lang.reflect.Modifier

Modifier se utiliza junto con Field para consultar las características de los campos encontrados mediante reflexión. En este proyecto permite comprobar los modificadores asociados a los campos de sym.java y determinar cuáles corresponden a constantes utilizadas para representar tokens. Se eligió porque facilita el filtrado de la información obtenida dinámicamente y evita procesar campos que no corresponden a símbolos del lenguaje.

java.nio.charset.StandardCharsets

StandardCharsets proporciona las codificaciones de caracteres estándar disponibles en Java. En este proyecto se utiliza específicamente UTF-8 para leer y

escribir correctamente los archivos que contienen el código fuente y los resultados del análisis. Se eligió porque el lenguaje utiliza símbolos especiales como λ , θ , Σ , ζ , $?$, ϵ , ω , $\int$, η y otros caracteres que deben conservarse correctamente durante el procesamiento.

java.nio.file.Path

Path representa una ruta dentro del sistema de archivos. En este proyecto se utiliza para almacenar y manipular las ubicaciones de los archivos fuente, archivos de salida y otros recursos necesarios para la ejecución. Se eligió porque permite trabajar con las rutas de una manera estructurada y facilita la construcción de ubicaciones de archivos independientemente de las operaciones realizadas sobre ellos.

java.nio.file.Paths

Paths se utiliza para crear objetos Path a partir de rutas escritas como texto. En este proyecto permite convertir las ubicaciones de los archivos del proyecto en objetos que pueden ser utilizados por las clases de java.nio.file. Se eligió porque simplifica la construcción y manipulación de las rutas necesarias para acceder a los archivos de entrada y salida.

java.util.ArrayList

ArrayList permite almacenar una colección dinámica de elementos. En este proyecto se utiliza para almacenar información que puede aumentar durante la ejecución, como errores encontrados durante el análisis. Se eligió porque permite agregar elementos dinámicamente sin tener que conocer previamente la cantidad exacta de elementos que se almacenarán.

java.util.HashMap

HashMap permite almacenar información utilizando una estructura de clave y valor. En este proyecto se utiliza para relacionar diferentes elementos del análisis, por ejemplo, información asociada con tokens, símbolos o nombres. Se eligió porque permite realizar búsquedas rápidas mediante una clave y resulta apropiado cuando se necesita establecer una relación entre dos datos.

java.util.LinkedHashSet

LinkedHashSet permite almacenar elementos sin duplicados y, al mismo tiempo, conservar el orden en que fueron agregados. En este proyecto se utiliza cuando es necesario mantener conjuntos de información sin repetir elementos y conservar un orden determinado. Se eligió sobre un Set convencional cuando también es importante mantener el orden de inserción de los elementos.

java.util.regex.Matcher

Matcher se utiliza junto con Pattern para buscar coincidencias de expresiones regulares dentro de cadenas de texto. En este proyecto permite analizar determinados patrones de texto durante el procesamiento de la información generada por el sistema. Se eligió porque proporciona mecanismos integrados en Java para realizar búsquedas y validaciones basadas en patrones.

java.util.regex.Pattern

Pattern permite definir y compilar expresiones regulares que posteriormente pueden ser utilizadas mediante Matcher. En este proyecto se utiliza para establecer patrones que deben ser identificados dentro de diferentes cadenas de texto. Se eligió porque las expresiones regulares permiten realizar búsquedas y validaciones de texto de manera precisa sin implementar manualmente cada comparación.

java_cup.runtime.Symbol

Symbol pertenece al runtime de Java CUP y es una de las clases fundamentales para la comunicación entre el analizador léxico y el analizador sintáctico. En este proyecto se utiliza en Lexer.flex para crear los símbolos que representan los tokens reconocidos y enviarlos al parser generado por CUP. Cada símbolo puede contener información como el tipo de token, el lexema, la línea y la columna. Se eligió porque es la estructura proporcionada por CUP específicamente para representar los símbolos utilizados durante el análisis sintáctico y permite conectar directamente el lexer generado por JFlex con el parser generado por CUP.

JFlex 1.9.1

JFlex es la herramienta utilizada para generar automáticamente el analizador léxico del proyecto. Se utiliza a partir del archivo `Lexer.flex`, donde se encuentran definidas las reglas para reconocer palabras reservadas, identificadores, literales, operadores, delimitadores, comentarios y errores léxicos. Se eligió JFlex porque está diseñado específicamente para generar analizadores léxicos en Java y permite definir los patrones del lenguaje mediante expresiones regulares, reduciendo la necesidad de implementar manualmente el reconocimiento carácter por carácter.

Java CUP 11b

Java CUP es la herramienta utilizada para generar el analizador sintáctico del proyecto. Se utiliza a partir del archivo `Parser.cup`, donde se encuentran las producciones que describen la gramática del lenguaje. Se eligió Java CUP porque permite definir formalmente la estructura sintáctica del lenguaje y generar automáticamente el parser encargado de comprobar si los tokens producidos por JFlex cumplen con las reglas establecidas. Además, proporciona mecanismos para manejar errores sintácticos y realizar recuperación mediante el símbolo especial error.

Java JDK 21.0.12

El JDK de Java es el entorno utilizado para compilar y ejecutar todo el proyecto. Se utiliza para compilar `Main.java`, `Lexer.java`, `parser.java` y `sym.java`, además de proporcionar las librerías estándar utilizadas por el programa. Se eligió Java porque es el lenguaje requerido para el proyecto y porque permite integrar directamente las herramientas JFlex y Java CUP, además de proporcionar las clases necesarias para el manejo de archivos, estructuras de datos y procesamiento de texto.

Análisis de resultados**Objetivos alcanzados**

- Implementación del analizador léxico con JFlex, permitiendo reconocer palabras reservadas, identificadores, tipos de datos, literales, operadores, delimitadores, comentarios y espacios en blanco.
- Generación del archivo de tokens, registrando los tokens y lexemas reconocidos durante el análisis léxico.
- Implementación del analizador sintáctico con Java CUP, capaz de validar declaraciones, asignaciones, expresiones, condicionales, ciclos, funciones, parámetros, entradas, salidas, retornos y el procedimiento principal.
- Definición de precedencia y asociación de operadores, organizando las expresiones en diferentes niveles para respetar el orden de las operaciones y evitar ambigüedades.
- Detección de errores léxicos, incluyendo identificadores inválidos, literales mal formados, cadenas o caracteres sin cerrar, números no permitidos y símbolos desconocidos.
- Detección y recuperación de errores sintácticos mediante Panic Mode, utilizando el mecanismo error de CUP y puntos de sincronización para continuar el análisis después de encontrar errores.
- Automatización del proceso de compilación y ejecución, mediante archivos .bat que permiten generar JFlex y CUP, compilar el proyecto y ejecutar el analizador sobre archivos .cmm.
- Integración del análisis léxico y sintáctico, permitiendo procesar un programa. cmm de forma automática y verificar su cumplimiento con las reglas definidas para el lenguaje.

Justificación de toma de decisiones

Para definir las expresiones de la gramática se decidió utilizar varias producciones separadas en diferentes niveles, en lugar de utilizar una única producción general que permitiera combinar todos los operadores de forma indistinta. Esta decisión se tomó porque las expresiones del lenguaje contienen diferentes tipos de operadores y estos no deben tener toda la misma prioridad al momento de ser analizados. La separación de las producciones permite representar

de forma clara la estructura de una expresión y hacer que el analizador sintáctico interprete los operadores en el orden establecido por el lenguaje.

Las producciones de expresiones se organizaron de manera jerárquica para representar los diferentes niveles de precedencia. De esta forma, los operadores que deben evaluarse primero se encuentran en niveles más internos de la gramática, mientras que los operadores con menor prioridad se encuentran en niveles superiores. Esta organización permite que una expresión como una combinación de operaciones aritméticas sea interpretada de acuerdo con las reglas definidas por el lenguaje, sin depender de una interpretación ambigua por parte del parser.

La decisión de separar las expresiones también permite controlar correctamente la asociación de los operadores. Los operadores que representan operaciones que se realizan de izquierda a derecha pueden definirse mediante producciones recursivas a la izquierda, mientras que aquellos que requieren otro comportamiento pueden establecerse mediante la estructura correspondiente. Esto permite que expresiones con varios operadores consecutivos mantengan un orden de análisis consistente y evita que existan diferentes árboles sintácticos posibles para una misma expresión.

En el caso de los operadores aritméticos, la gramática diferencia los niveles correspondientes a las operaciones de menor y mayor prioridad. Por ejemplo, las operaciones de suma y resta no se encuentran en el mismo nivel que las operaciones de multiplicación, división y módulo. Esto permite que una expresión que combine diferentes operaciones sea reconocida respetando la prioridad matemática esperada. De esta manera, una expresión que contenga una suma y una multiplicación no se interpreta simplemente de izquierda a derecha, sino que la multiplicación queda asociada al nivel de mayor precedencia antes de considerar la suma.

También se decidió manejar la potencia mediante un nivel específico dentro de las producciones de expresiones. Esto es importante porque la potencia tiene una prioridad diferente respecto de las operaciones aritméticas básicas y, además,

puede combinarse con expresiones agrupadas. Al disponer de una producción específica para este operador, la gramática puede reconocer construcciones como una base seguida de una potencia cuyo exponente también sea una expresión, manteniendo separada esta operación del resto de los operadores aritméticos.

El uso de producciones independientes para cada nivel también facilita el tratamiento de los paréntesis y de las expresiones agrupadas definidas por el lenguaje. Una expresión encerrada dentro de los delimitadores correspondientes puede ser analizada nuevamente como una expresión completa, permitiendo modificar el orden natural establecido por la precedencia. Esto hace que la gramática no solo reconozca expresiones simples, sino también expresiones compuestas donde el programador necesita indicar explícitamente qué operación deben analizarse primero.

Otra razón para utilizar esta estructura es reducir los conflictos que pueden aparecer en el analizador sintáctico de CUP. Si todos los operadores se definieran mediante una única producción del tipo expresión-operador-expresión, el parser podría encontrar diferentes posibilidades para interpretar una misma secuencia de tokens. Al dividir la expresión en niveles, la propia estructura de la gramática establece qué operadores pueden aparecer en cada posición y cuál debe ser su prioridad, reduciendo la ambigüedad durante el análisis sintáctico.

Bitácora

https://github.com/HeldyisAE/PY1_CompiladoresEInterpretes/commits